Final Project Design Phase

Computer Science - Week 11

Oct 3, 2026 - Version 0.0.4

For complete instructions and project ideas, refer to the assignment in Canvas. Specifically, review the final project requirements before you continue below. If you have not reviewed the project requirements in Canvas (you might have to go to the modules view and check the current or past week's content), you will not be able to properly complete the parts below. In addition, it may be helpful to review the information on this Drafter documentation page.

0) Project Planning and Preparation

Successful programmers spend time planning before they begin coding. *After* reviewing the project requirements, answer the following questions to help guide your design and development process.

A. What concepts from class or previous assignments might help you complete this project? Which programming concepts (loops, conditionals, lists, classes, etc.) do you expect to use? What programming skills do you want to improve while completing this project?

I plan on using many concepts from class and previous assignments such as loops, conditionals, lists, dataclasses, and overall coding techniques such as debugging or reducing tech debt. I expect to use loops to filter through plants, conditionals to check if a plant has been watered, lists to keep a collection of the user's plants, and dataclasses to store multiple pieces of information for each plant. I want to improve my debugging skills while completing this project, as a lot of times I might not have trouble writing code, but if it doesn't work, I find myself struggling to fix it.

B. Which parts of the project do you expect to be the easiest for you? The hardest? Why?

I think that having the user add information for a new plant will be the easiest because I remember seeing a drafter example website similar to this, and it seems like pretty simple appending and updating variables. I think that the hardest part of the project will be how I am going to display all of the information to the user, because it sounds like some complex organization that I am not too familiar with in drafter yet. I also think that it will be hard to connect the pages and routes together because, again, I am not too familiar with using that many different pages and needing routes to

connect them all.

C. Even though you have not yet designed the final project, estimate which tasks will likely require the most time and effort? Which features or components should be implemented first?

I think that the pages will take the most time and effort because I need to figure out exactly what information I want on them and how to display that information in a way that I find presentable and I need to figure out how to connect them all with routes to make the website flow. I think that I should implement the State dataclass and the list of Plant dataclasses that will go inside of it first just to get everything organized and be able to see every piece of information that there will be.

D. What distractions might interfere with your work and how will you minimize them? What resources will you need available while working? What questions, if any, do you still have about the assignment requirements?

My phone or a show might distract me and interfere with my work. I will minimize these distractions by making sure I am not listening to anything too distracting and keeping my phone away from me to take away the urge to just grab it. I plan to use a LLM to code my website, so I will need that, and I also find that quiet music and noise canceling headphones helps me focus, so I will use that while working. It will also be helpful to look at my plans and mock-up made in this assignment while working. I do not have any questions about the assignment requirements.

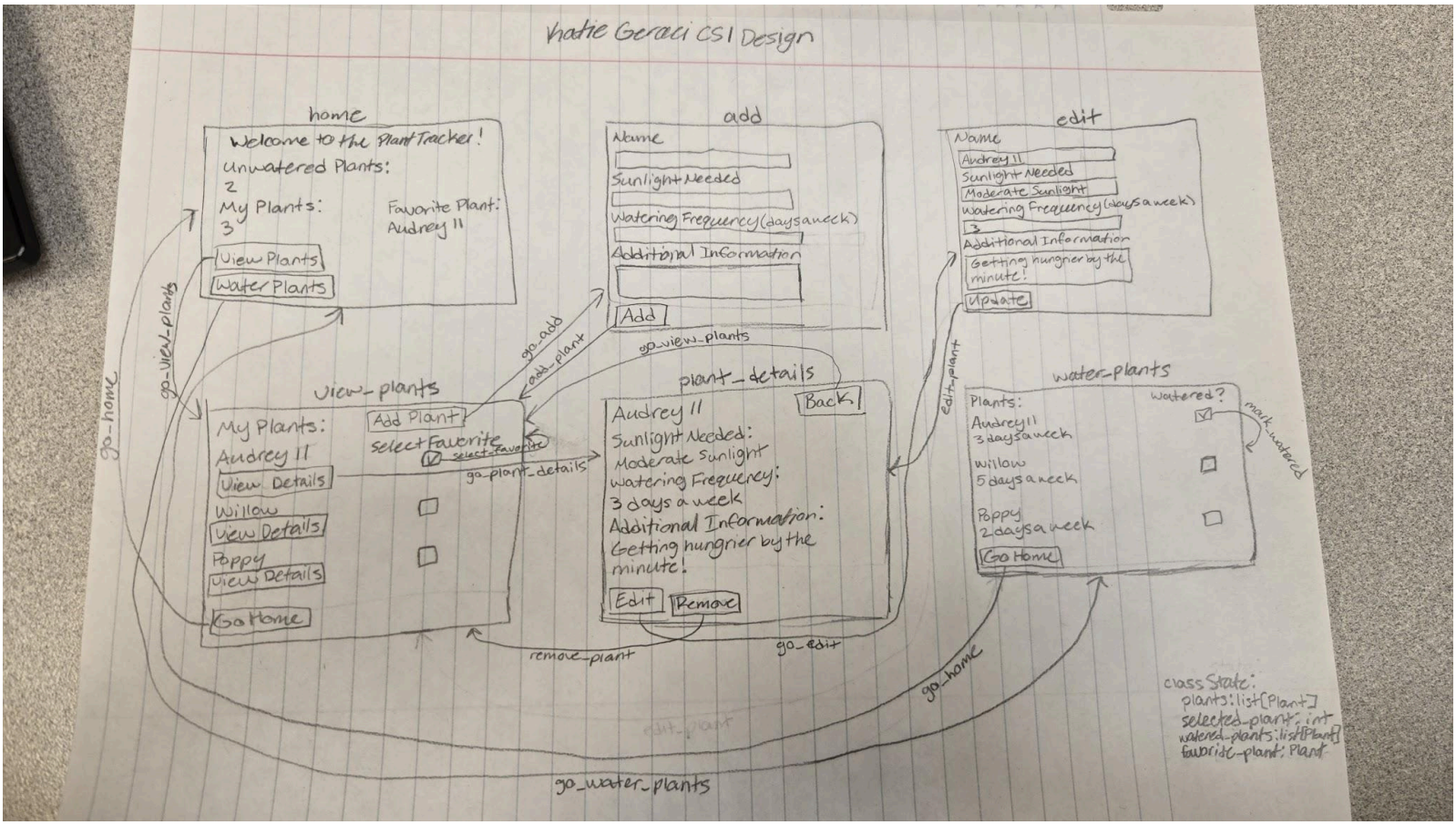
1) Site Diagram

Create a mock-up of your planned website. We suggest you make the mockup on paper, although you can also use software like PowerPoint or Slides to make a mockup. There is other design software that could also be used, but we strongly encourage you to keep it simple. Do not spend more than 15-30 minutes detailing the mock-up. Your goal should be to make a very quick and dirty diagram, to show that you have some kind of idea.

We recommend drawing each page as a rectangle, and laying out the text, buttons, and other components inside. Then, use arrows to indicate links between pages. Label each page the way you will name each function, and then put labels on the arrows to indicate what the button links. Also write down any parameters that should come into the pages.

In addition to the pages and routes, write down what fields your `State` will have, both their names and types. Think clearly about what data will need to be persistent across the entire website.

Add photo(s) or screenshot(s) of your website mock-up and labeled diagram on the next page.



2) Implementing Requirements

This table should serve as a draft for how you will implement the requirements into your final project. You may change your ideas later on, but your final site must fulfill all the requirements.

Requirement	**Implementation**
Purpose/Functionality	The purpose of my website will be for users to input their plants and information about them, such as how much sun they need or how often they need to be

A 1-2 sentence description of the purpose or functionality of your site	watered, so that they are able to keep track of the care needs of their plants.
State Dataclass Attributes Minimum of 4 attributes, one of which must be a list/dict of instances of your second dataclass. Briefly describe each attribute, its type, and how it will be meaningfully modified throughout the site.	State: plants: list[Plant] selected_plant: int watered_plants: list[Plant] favorite_plant: Plant The plants attribute has a list of Plant dataclasses which will represent the user's list of plants. It will be modified to add, delete, or edit the user's plants. The selected_plant attribute has an integer representing the index of a plant in the user's list of plants. This will be modified as users will be able to select different plants to see their information. The watered_plants attribute has a list of Plant dataclasses and will be modified when a new plant has been watered, adding it to the list. This will make it more helpful to count the number of watered plants for the home page. The favorite_plant attribute will store a plant selected as the favorite plant. It will be modified as the user selects which plant they want to be their favorite, updating the stored favorite every time a different favorite is selected. This will be used to display on the home page.
Second Dataclass Your second dataclass, including its attributes, their types, and how they will be meaningfully modified (if they are modified)	Plant: name: str sunlight: str water_frequency: int is_watered: bool additional_information: str They will all be able to be modified in the sense that the user will be able to select one of their plants and choose to edit them if desired, changing some of the

	information about them. Also, is_watered is different, as it will be modified with a checkbox button that the user can press to indicate if they have or have not watered their plant.
Routes (minimum 7) The names of your route functions	go_home go_add go_view_plants go_edit go_plant_details go_water_plants add_plant edit_plant remove_plant mark_watered select_favorite
Pages (minimum 5) A brief description of the pages in your site	home The home page will have a welcome message and show how many plants are in the user's collection and how many need to be watered. add The add page will allow the user to input any information in the given boxes about a plant that they want to add to the website. By clicking "Add", the new plant that they have filled out will be added to their list of plants. edit The edit page will look very similar to the add page, but since the user is editing an already existing plant, all of their information will stay in the textboxes and they will be able to change what is already there as they please. view_plants

	The view_plants page will be an area just for looking at all of the plants in the user's collection. Each plant will have a button that, when clicked, will lead to another page with all of the plant's information. plant_details The plant_details page will have all of the information about a certain plant selected. This is also where the user will be able to choose to edit or delete the plant. water_plants The water_plants page will have each plant and their watering frequency. Here, the user will be able to check a box to say if they have watered their plant or not for each plant.
User Input (minimum 3) Description of user input components (TextBox, TextArea, CheckBox, or SelectBox) and their purposes For more information on these input types, see Drafter Documentation.	A CheckBox that will allow users to check off if a plant has been watered, updating is_watered for that plant. A TextBox for the user to input the name of their plant. A TextBox for the user to input how much sun the plant needs. A TexBox for the user to input how many days their plant needs to be watered a week. A TextArea for the user to input any additional information about their plant. A TextBox for the user to edit their plant name. A TextBox for the user to edit how much sun is needed for their plant. A TextBox for the user to edit how many days their plant needs to be watered a week. A TextArea for the user to edit their plant's additional information.
If Statements (minimum 3) Brief description of what will be evaluated in your if statements and how they will meaningfully	An if statement to check if a plant has been watered. This is needed to know if the plant has been watered or not and will indicate to the user how many plants have been watered or not, which is an essential

change the logic of the site	component to the tracker. An if statement to check if the list of plants is empty. This is needed because a user will not be able to see their plants if there are none to begin with, so there must be an if statement to indicate if there are or are not plants to be seen. An if statement for each TextBox to make sure there is not an empty string (“”). This is needed because a user will need to input at least something, for example, the plant’s name, for the tracker to really work. An if statement for the TextBox for the sunlight and watering frequency to make sure the user imputed a number. This is needed because, if they didn’t, the code will not be able to properly use what they imputed. An if statement for the TextBox for the plant’s name to make sure the name does not already exist for an already existing plant. This is needed because the user should not be able to add two different plants with the same name, or have two different tabs for the same plant.
Loop (minimum 1) This loop must iterate over the list/dict of dataclass instances in your State attribute. Description of this loop and why it is meaningful to the site	This loop will iterate over the list of Plant. This means it will check every plant in plants (the collection of the user’s plants). It is very meaningful to the site because each individual plant needs to be accessed in order for the user to observe each piece of information about each individual plant.

LLM Conversation

If you used an LLM in any part of this work, copy and paste every part of that conversation below. Otherwise, you may be given a zero for the assignment. Obviously, these words will not all be yours, and this is the exception to the requirement at the top of this assignment that all responses be in your own words. If you did not use an LLM to assist you in this assignment, simply say so below.

I am making a website where the purpose will be for users to input their plants and information about them, such as how much sun they need or how often they need to be watered, so that they are able to keep track of the care needs of their plants. There are requirements for this project and I am having some trouble thinking of ways to fulfil them. Can we work through them together?
Absolutely! I think a plant-care tracker is a great project idea for a CS1 final project. It has a clear purpose, plenty of opportunities for creative features, and can be built using relatively simple Python programming concepts.
I've also looked through your uploaded CS1 Final Project Alpha worksheet, so we can work through the actual requirements rather than just brainstorm features.
First, let's establish the scope of your website
Your basic idea is:
A website where users can add plants to a personal collection, record each plant's care requirements, and use the website to keep track of when and how their plants need attention.
Here are some features we could consider:
Golden Pothos plant ceramic pot | Free Photo - rawpixel
1. Plant collection
Users can add plants, give them names, and view all their plants in one place.
Core feature
Regadera tipo ducha – Vitali+
2. Watering tracker
Store how often each plant needs watering and when it was last watered.
Core feature
Best Houseplants That Thrive in Low Humidity
3. Sunlight requirements
Record whether a plant needs direct sunlight, indirect sunlight, or shade.
Jak prowadzić dziennik pielęgnacji roślin - GardenCasa blog o roślinach doniczkowych
4. Plant care dashboard
Display upcoming watering needs, plant information, and plants that need attention.
A few things to keep in mind technically
Since you're using Drafter, we should design around what it can realistically do.
Saving plants: Use browser local storage so plant information remains available after refreshing the page.
Adding and editing plants: Use forms and buttons.
Tracking watering: Store the last-watered date and calculate how many days have passed.
Displaying information: Use lists, cards, tables, and possibly graphs.
Reminders: Display reminders inside the website rather than relying on background notifications.
We should avoid features requiring user accounts, a backend database, or complicated server functionality.
Let's work through your worksheet together
I suggest we go section by section, starting with Part 1: The Problem & Prior Solutions.
I want to understand a little more about your vision before we start writing answers.
First question: How do you imagine someone using your website?
For example, would you prefer:
A personal plant journal: Users manually add their plants and their care instructions.
A plant-care assistant: Users add plants, and the website helps them figure out what needs watering or attention.
A combination: Users maintain a plant collection, record care information, and receive reminders about what needs to be done.
My initial recommendation is option 3, because it gives your application a clear purpose while still being achievable for a three-week CS1 project.
We can develop the idea together, and I'll help you turn each requirement into an answer that makes sense for your project without making it unnecessarily complicated.
I need help thinking of State dataclass attributes. So far, I know that I need a list of Plant dataclasses where the Plan dataclass has attributes sunlight: int, water_frequency: int, is_watered: bool, and additional_information: str. However, the State dataclass needs a minimum of 4 attributes and I need help thinking of more.
I would recommend making your State dataclass represent the overall website, while your Plant dataclass represents the information about an individual plant.
You already have one important State attribute:
plants: list[Plant]
Here are some additional attributes that would make sense for your application.
Suggested State dataclass attributes
Attribute
Data type
Purpose
plants
list[Plant]
Stores all plants added by the user.
selected_plant
int
Keeps track of which plant the user is currently viewing.
new_plant_name
str
Stores the name entered when the user adds a plant.
search_query
str

Stores text entered into a search bar to find a specific plant.
current_page
str
Keeps track of which page of the website the user is viewing.
You only need four attributes, so you can choose the ones that make the most sense for your design.
My recommended combination
I would personally choose these four:

```
@dataclass
class State:
    plants: list[Plant]
    selected_plant: int
    search_query: str
    current_page: str
```

Here's why I like this combination:
plants allows the user to maintain their entire plant collection.
selected_plant allows the user to select an individual plant and view its information.
search_query allows the user to search through their collection if they have many plants.
current_page allows you to manage navigation between different sections of your website.
One additional suggestion for your Plant dataclass
I noticed that you mentioned your Plant dataclass has these attributes:
sunlight: int
water_frequency: int
is_watered: bool
additional_information: str
I would strongly recommend adding:
name: str
This is important because users need to be able to identify which plant they're looking at!
You could also eventually add a last_watered attribute if you want to calculate when a plant needs watering again.
One thing I'd like to ask: Are you planning on allowing users to add, remove, and edit plants after they've initially entered them? If so, I can help you think through how to structure your State and Plant dataclasses so those features work smoothly in Drafter.
I plan on allowing users to add, remove, and edit plants after initially entering them. How would your suggested attributes be used in practice? I am having trouble visualizing them in my website.
I think the easiest way to understand the State attributes is to imagine what happens when someone actually visits your website and interacts with it.
Let's walk through a possible user experience and connect each attribute to something the user would see.
1. What your website might look like
Imagine the user opens your website and sees a page called My Plants.
My Plant Collection 🌱
Keep track of your plants and their care needs.
My Plants (3)
3 plants
Monstera plant leaf white background | Free Photo - rawpixel
Monstera
Water every 7 days
Watered
Echeveria | Painted Lady
– Pigment
Succulent
Water every 14 days
Needs water
Epipremnum aureum | Golden Pothos | Foliage Factory
Pothos
Water every 5 days
Watered
Illustrative website layout
Now let's look at how your State attributes would help make this website function.
2. Understanding each State attribute
Attribute 1: plants: list[Plant]
This is probably the most important attribute.
It stores all the plants the user has entered.
For example:

```
plants = [
    Plant("Monstera", 6, 7, True, "Likes indirect sunlight"),
    Plant("Succulent", 8, 14, False, "Needs bright sunlight"),
    Plant("Pothos", 4, 5, True, "Easy to maintain")
]
```

Whenever a user adds a plant, it gets added to this list.
Whenever they remove a plant, it gets removed from this list.
Whenever they edit a plant, the information in this list gets updated.

Attribute 2: selected_plant: int
This keeps track of which plant the user has selected.
Imagine the user clicks on their Monstera. Your website could open a separate page showing its information:
MY PLANTS / PLANT DETAILS
Selected
Five Large-Leaf Indoor Plants to Bring Life and Character to Homes and Offices
Monstera
Sunlight: 6 hours
Water frequency: Every 7 days
Watered: Yes
Additional information: Likes indirect sunlight.
Your selected_plant attribute could store the index of the selected plant.
For example:
selected_plant = 0
This means the first plant in your list is selected.
Why is this useful?
It allows your website to know which plant the user wants to edit or remove.
Attribute 3: search_query: str
Imagine the user eventually has 20 plants.
Instead of scrolling through all of them, they can search for a specific plant.
Find a plant
Search your plants...
United Nursery Swiss Cheese Plant Monstera Deliciosa Plant 24. in to 34 in. Tall in 10 in. White Decor Pot 74639 - The Home Depot
Monstera
Search result
The search_query attribute stores whatever the user types.
For example:
search_query = "Monstera"
Your program then searches the plants list and displays plants whose names match the search.
Why is this useful?
It makes your website easier to navigate as the user's collection grows.
Attribute 4: current_page: str
Imagine your website has three different pages:
Home page: Displays the plant collection.
Add Plant page: Contains a form for entering a new plant.
Plant Details page: Displays information about an individual plant.
Your current_page attribute tells Drafter which page the user should currently see.
For example:
current_page = "home"
When the user clicks Add Plant:
current_page = "add_plant"
When they select an existing plant:
current_page = "plant_details"
Why is this useful?
It helps you manage navigation throughout your website.
3. How the attributes work together
Here's a simplified example of how a user might interact with your website:
User opens the website
current_page = "home"
plants = []
User adds a Monstera
The Monstera is added to plants.
plants = [Monstera]
User clicks on the Monstera
selected_plant = 0
current_page = "plant_details"
User edits its watering frequency
The Monstera's water_frequency is updated from 7 to 10.
User returns to the home page
current_page = "home"
The updated information is displayed.
4. My recommendation: You might not need all four!
One important thing about State dataclasses is that you should only include attributes that your program actually needs.
For your particular project, I would consider this alternative:
State attribute
Purpose
plants: list[Plant]
Stores the user's plants.

current_page: str
Controls website navigation.
selected_plant: int
Identifies the plant being viewed or edited.
new_plant_name: str
Temporarily stores the name entered in the Add Plant form.
I actually like this combination better if you are planning to focus on adding, editing, and removing plants rather than building a complicated search feature.
One final thought: Since you're using Drafter, remember that State attributes are not necessarily permanent storage. You will want to consider local storage separately if you want users' plants to remain saved when they close and reopen the website.
My suggestion is to build around these four attributes first. You can always add a search feature later if you have time!
Now I need help finding a minimum of 5 pages for my website. So far I have "
home
The home page will just have all of the buttons that can direct the user to all of the pages that they might want to reach.
add
The add page will allow the user to input any information in the given boxes about a plant that they want to add to the website. By clicking “Add”, the new plant that they have filled out will be added to their list of plants.
view_plants
The view_plants page will be an area just for looking at all of the user's plants and all of the information that goes along with them, including watering frequency, sunlight needed, and additional information. It is on this page that the user will be able to choose if they want to add or edit a plant if desired.
water_plants
The water_plants page will have each plant and their watering frequency. Here, the user will be able to check a box to say if they have watered their plant or not for each plant."
However, I am open to more suggestions and changing some of what I have now that we have thought of State dataclasses. Can you please help me?
…
Absolutely! I think your first four pages are a really good starting point. I would actually keep most of what you have and build the remaining pages around the features you've already planned.
One thing I want to emphasize is that each page should have a distinct purpose. You don't want to create five pages that all do essentially the same thing just to meet the requirement.
Here is how I would organize your website.
My suggested 6-page website
1. home — Home page
Plant Care Dashboard UI by Ismail Nahid on Dribbble
Your description is already good!
The home page will serve as the main navigation area of the website. It will contain buttons that direct users to the different pages of the website.
You could also include:
A welcome message.
The total number of plants in the user's collection.
A small reminder showing how many plants need watering.
State attributes used: current_page, plants
2. add — Add a plant page
Houseplant Journal - Printable Plant Planner, Plant Watering Schedule Log, Indoor Plant Care Tracker - Etsy
I would keep this page almost exactly as you described it.
Users will be able to fill out a form containing information about a new plant.
Possible input fields:
Plant name.
Sunlight requirements.
Watering frequency.
Whether the plant has already been watered.
Additional information.
When the user clicks the Add button, the plant will be added to their collection.
State attributes used: plants, current_page
3. view_plants — View all plants page
ChooseYourPlant, una nueva red social para amantes de las plantas
I would keep this page, too, but make one small adjustment.
Rather than allowing users to edit plants directly on this page, I would have each plant displayed as a card with a button to view its details.
For example:
Monstera — View Details
Snake Plant — View Details
Pothos — View Details
This keeps the page organized and gives you a reason to create another page.
State attributes used: plants, selected_plant, current_page
4. plant_details — Individual plant details page
Set Reminders Page - Plant Care App by Nastassia Kashura on Dribbble
My recommended new page

This page would open when the user selects a particular plant from their collection.
It would display all the information associated with that plant.
For example:
Plant name.
Sunlight requirements.
Watering frequency.
Whether it has been watered.
Additional information.
You could also include two buttons:
Edit Plant: Takes the user to the edit page.
Remove Plant: Removes the selected plant from their collection.
This page gives your selected_plant State attribute a very clear purpose!
State attributes used: plants, selected_plant, current_page
5. water_plants — Watering tracker page
Plant Watering Tracker, House Plant Care Tracker, Watering Schedule, Monthly Plant Watering Checklist, Plant Care, Watering Log, A4 US - Etsy
I really like this idea, and I would definitely keep it.
This page will display all the user's plants along with their watering frequencies.
Users can check a box to indicate whether they have watered a particular plant.
For example:
Plant
Watering frequency
Watered?
Monstera
Every 7 days
Yes
Snake Plant
Every 14 days
No
Pothos
Every 5 days
Yes
You could also include a button to mark all plants as watered, although that is entirely optional.
State attributes used: plants, current_page
One thing to consider: Your current is_watered Boolean only records whether a plant is marked as watered. If you eventually want to calculate when the next watering is due, you would need an additional attribute such as last_watered.
6. edit_plant — Edit a plant page
Nursery Plant Care App UI :: Behance
My recommended new page
Since you mentioned that users should be able to edit their plants, I think giving this its own page is a great idea.
This page would contain a form similar to the Add Plant page.
However, instead of starting with empty input boxes, the boxes would already contain the information associated with the selected plant.
For example, if the user selects their Monstera, the form would already contain:
Name: Monstera
Sunlight: 6 hours
Water frequency: 7 days
Additional information: Likes indirect sunlight.
The user can change any information and click Save Changes.
The updated information would replace the old information in the plants list.
State attributes used: plants, selected_plant, current_page
How the pages connect
I think this is one of the most important things to plan before you start programming.
Here is a possible navigation structure:
HOME
Main navigation
ADD
Create a plant
VIEW PLANTS
See collection
WATER PLANTS
Track watering
PLANT DETAILS
View selected plant
EDIT PLANT
Update selected plant
Navigation example, not every page needs to connect directly to Home.
Final list of pages
Here is the list I would use in your project proposal:

Page name
Main purpose
home
Main navigation and overview of the website.
add
Allows users to add new plants to their collection.
view_plants
Displays all plants currently in the user's collection.
plant_details
Displays detailed information about an individual plant.
water_plants
Allows users to track which plants have been watered.
edit_plant
Allows users to modify information about an existing plant.
My recommendation: I would go with these six pages rather than trying to come up with five unrelated features. They all contribute to the same overall purpose of your website.
One other benefit is that these pages give you a straightforward way to demonstrate your CS1 programming skills: lists, dataclasses, Boolean values, conditionals, loops, forms, and functions.
You have a project that can be both useful and realistic to complete in three weeks!
I like this plan a lot, thank you! Can you help me think of at least 7 route functions to connect these pages?
Absolutely! Since you're using Drafter, your route functions will be responsible for navigating between pages and updating the State when necessary.
I would aim for 8 route functions so you have more than the required 7 and can connect all six of your pages.
Here are the ones I recommend.
Suggested route functions
Route function
Purpose
go_home
Takes the user to the home page.
go_add
Takes the user to the Add Plant page.
add_plant
Adds the newly entered plant to the plant list and returns the user to the collection page.
go_view_plants
Takes the user to the View Plants page.
view_plant_details
Displays the details of the plant selected by the user.
go_water_plants
Takes the user to the Water Plants page.
go_edit_plant
Takes the user to the Edit Plant page for the selected plant.
edit_plant
Saves changes made to an existing plant and returns the user to its details page.
remove_plant
Removes the selected plant from the user's collection and returns them to the collection page.
That's actually 9 functions, which gives you some flexibility!
How they would work together
Here's a visual representation of how users would navigate your website.
HOME PAGE
go_home
ADD PAGE
go_add
add_plant
VIEW PLANTS
go_view_plants
view_plant_details
WATER PLANTS
go_water_plants
PLANT DETAILS
go_edit_plant
edit_plant / remove_plant
A few important distinctions
I want to point out something that will help you when programming in Drafter.
There are two different types of route functions in your project:
1. Navigation routes
These simply change which page the user sees.
Examples:
go_home
go_add

go_view_plants
go_water_plants
go_edit_plant
These generally update:
state.current_page = "home"
2. Action routes
These actually manipulate information in your State.
Examples:
add_plant
edit_plant
remove_plant
For example, add_plant adds a new Plant dataclass to state.plants, while remove_plant removes an existing one.
You can also combine navigation and data manipulation in the same function. For example, add_plant can add the plant and then change the current page to "view_plants".
One more route function I recommend
Since you have a watering tracker, I would consider adding:
mark_watered
This function would update the is_watered Boolean for whichever plant the user selects.
For example:
plant.is_watered = True
This would be called when the user checks the box next to a plant on the Water Plants page.
My recommendation I would aim for these 10 functions:
go_home
go_add
add_plant
go_view_plants
view_plant_details
go_water_plants
mark_watered
go_edit_plant
edit_plant
remove_plant
These give every page a purpose and provide all the basic functionality your website needs.
You don't necessarily need to implement all ten immediately. I would start with navigation routes, then work on adding, editing, and removing plants, and finally implement the watering tracker.
Now I need at least 3 user input components. I think I need a lot for this website because it relies a lot on user inputs but I need help identifying all of them. So far I have a CheckBox that will allow users to check off if a plant has been watered, updating is_watered for that plant. Can you help me please?
Absolutely! Your website is actually a great candidate for user input components because almost every feature involves users entering, selecting, or updating information about their plants.
I would aim for at least 8–10 different input components or uses of input components. You don't necessarily need that many different types of components; you can reuse the same type in different places.
Here is how I would organize them by page.
1. Add Plant page
This page will probably have the most user inputs because users need to provide all the information about their new plants.
Add a New Plant 🌱
1. Text Input — Plant Name
Enter your plant's name...
Stores the plant's name in a string.
2. Number Input — Sunlight Requirements
Hours of sunlight needed
Stores the number of sunlight hours needed (integer).
3. Number Input — Watering Frequency
Days between watering
Stores how often the plant needs watering.
4. Checkbox — Already Watered?
I have already watered this plant.
Updates the is_watered Boolean.
5. Textarea — Additional Information
Enter any additional information about your plant...
Stores additional information as a string.
That's already five user input components!
2. View Plants page
You could include another input component here.
6. Search Bar — Find a Plant
Search your collection
Search by plant name...
Filters the plant list based on what the user types.
This would connect nicely to your search_query State attribute that we discussed earlier.

For example, if the user types "Monstera", your website would display only plants with names matching that search.
3. Water Plants page
You already identified a great input component for this page.
7. Checkbox — Mark Plant as Watered
Watering Checklist
Monstera
Watered
Snake Plant
Watered
Pothos
Watered
Each checkbox updates the corresponding plant's is_watered attribute.
An important detail: each checkbox needs to update the correct plant, not every plant in the list.
4. Edit Plant page
This page can reuse several of the input components from the Add Plant page.
8. Text Input — Edit Plant Name
Allows users to change the name of an existing plant.
9. Number Input — Edit Sunlight Requirements
Allows users to change how much sunlight their plant needs.
10. Number Input — Edit Watering Frequency
Allows users to change how often their plant needs watering.
11. Textarea — Edit Additional Information
Allows users to update their plant's additional information.
The difference is that these input components should already contain the plant's existing information when the user opens the page.
5. Complete list of user input components
Here is how I would document them for your project planning:
#
Component
Page
Purpose
1
Text Input
Add
Enter plant name.
2
Number Input
Add
Enter sunlight requirements.
3
Number Input
Add
Enter watering frequency.
4
Checkbox
Add
Indicate whether plant is already watered.
5
Textarea
Add
Enter additional information.
6
Text Input
View Plants
Search for a particular plant.
7
Checkbox
Water Plants
Mark a plant as watered.
8
Text Input
Edit Plant
Change plant name.
9
Number Input
Edit Plant
Change sunlight requirements.
10
Number Input

Edit Plant
Change watering frequency.
11
Textarea
Edit Plant
Change additional information.
6. Two additional components you might consider
If you want to make your website a little more interesting, Drafter can also support other types of inputs.
Dropdown / Select
You could let users select sunlight requirements from options such as:
Full sun
Partial sun
Indirect sunlight
Shade
This might actually be better than asking users to enter a number of hours because sunlight descriptions are commonly used in plant care.
Different Types of House Plants: Light Explained
Radio Buttons
You could let users select whether a plant is:
An indoor plant
An outdoor plant
This could help users organize their collection.
How Often Should You Water Indoor and Outdoor Plants Easy Guide 2026
– IFFCO Urban Gardens
My biggest recommendation
I would actually consider changing your Plant dataclass slightly.
Currently, you have:
sunlight: int
water_frequency: int
is_watered: bool
additional_information: str
If you decide to use a dropdown for sunlight requirements, you could change sunlight to a string instead of an integer.
For example:
sunlight: str
This would allow users to store values like "Full Sun" or "Partial Shade" rather than a number.
I would prioritize these input types for your project:
Essential Text Input
Essential Number Input
Essential Checkbox
Essential Textarea
Optional Dropdown / Select
Optional Radio Buttons
You already have more than enough opportunities to meet the input requirement!
One important distinction: buttons such as Add, Save Changes, and Remove are interactive controls, but they are generally action controls rather than data-entry inputs. I would list them separately if your assignment specifically asks for user input components.
Which of these (TextBox, TextArea, CheckBox, or SelectBox) would they be?
Here is how I would categorize all the inputs for your plant-care website using the four Drafter input components you listed.
User input components
User input
Drafter component
Purpose
Plant name
TextBox
Enter the plant's name.
Sunlight requirements
SelectBox
Choose full sun, partial sun, indirect sunlight, or shade.
Watering frequency
TextBox
Enter the number of days between watering.
Already watered? (Add page)
CheckBox
Indicate whether the plant has been watered.
Additional information
TextArea
Enter notes about the plant.
Search for a plant
TextBox
Search the user's plant collection.

Mark plant as watered (Water Plants page)
CheckBox
Update the plant's is_watered attribute.
Edit plant name
TextBox
Change the plant's name.
Edit sunlight requirements
SelectBox
Change the sunlight selection.
Edit watering frequency
TextBox
Change watering frequency.
Edit additional information
TextArea
Update the plant's notes.
Summary of how many you have
TextBox
5 uses
TextArea
2 uses
CheckBox
2 uses
SelectBox
2 uses
My recommendation: Make sure you include at least one example of each of the four component types in your project.
One small distinction: These are 11 uses of input components, but only four distinct component types. If your professor requires three different types, you already exceed that requirement!
Can you help me think of at least 3 if statements that will be used?
Absolutely! Your plant-care website has several natural opportunities for if statements. I would recommend choosing conditions that actually help your website function rather than adding unnecessary ones just to meet the requirement.
Here are 7 possible if statements you could use.
1. Check whether a plant has been watered
Page: water_plants
If the user checks the watering checkbox, the program will update the plant's is_watered attribute to True. Otherwise, it will remain False.

```
if plant.is_watered:
    message = "This plant has been watered!"
else:
    message = "This plant still needs watering."
```

2. Check whether the user entered a plant name
Page: add
If the user leaves the plant name blank, the website should display an error message instead of adding the plant.

```
if plant_name == "":
    message = "Please enter a plant name."
else:
    # Add the new plant
```

3. Check whether the user has any plants
Page: view_plants
If the user's plant list is empty, display a message encouraging them to add their first plant.

```
if len(state.plants) == 0:
    message = "You haven't added any plants yet!"
else:
    # Display the user's plants
```

4. Check whether a plant needs watering
Page: water_plants
If a plant has not been watered, display a reminder to the user.

```
if plant.is_watered == False:
    message = "Remember to water this plant!"
else:
    message = "Your plant is all taken care of!"
```

This is similar to the first example, so you could combine these into one if statement rather than implementing both separately.
5. Check whether a plant exists before removing it
Page: plant_details
If the user has selected a valid plant, remove it from their collection. Otherwise, display an error message.

```
if state.selected_plant < len(state.plants):
    # Remove the selected plant
else:
    message = "Plant not found."
```

You should also check that the selected index is not negative.

6. Check whether a search produces results
Page: view_plants
If the user's search matches a plant, display the results. Otherwise, tell them no matching plants were found.

```
if len(search_results) == 0:
    message = "No plants found."
else:
    # Display matching plants
```

7. Check whether watering frequency is valid
Page: add
If the user enters a watering frequency of zero or a negative number, ask them to enter a valid number.

```
if water_frequency <= 0:
    message = "Please enter a valid watering frequency."
else:
    # Continue adding the plant
```

My recommendation: Which 3 should you use?
Top 3 choices
If I were choosing three for your project, I would prioritize:
If statement
Why I recommend it
Check whether the plant list is empty.
Makes your website more user-friendly.
Check whether the plant has been watered.
Essential to your watering tracker.
Check whether the plant name is blank.
Prevents incomplete plant entries.
These three are simple to understand, relatively easy to program, and directly related to your website's purpose.
One additional suggestion: You could also use an if statement to check whether a plant's watering frequency has passed based on its last-watered date. This would make your website more useful because it could distinguish between plants that have been watered recently and plants that are actually due for watering.
That would be a nice feature to work toward once you have the basic functionality working!
Can you help me understand how the state attributes current_page:str, selected_plant: int, and new_plant_name: str will be used and modified throughout the website again please?
Of course! I think these three attributes can be a little confusing because they all serve different purposes, and some of them are only useful on certain pages.
Let's go through each one using examples from your plant-care website.
1. current_page: str
Purpose: Keeps track of which page the user is currently viewing.
Think of this attribute as your website's navigation system. Whenever a user clicks a button to go somewhere else, current_page changes.
For example, your State might initially look like this:
current_page = "home"
Here is how it changes as the user navigates:
User action
New current_page value
Opens the website
"home"
Clicks Add a Plant
"add"
Clicks View Plants
"view_plants"
Clicks Water Plants
"water_plants"
Selects a plant
"plant_details"
Clicks Edit Plant
"edit_plant"
How it is modified:
Your navigation route functions will update this attribute.
For example, your go_add route would change:
state.current_page = "add"
Your go_home route would change:
state.current_page = "home"
Essentially, this attribute tells your program which page it should display.
2. selected_plant: int
Purpose: Keeps track of which plant the user has selected.
This attribute is particularly important for your View Plants, Plant Details, Edit Plant, and Remove Plant functionality.
Let's imagine the user has three plants stored in their plants list:
My Plants
United Nursery Swiss Cheese Plant Monstera Deliciosa Plant 24. in to 34 in. Tall in 10 in. White Decor Pot 74639 - The Home

Depot
Monstera
Index: 0
Snake Plant | Sansevieria Trifasciata Laurentii | Indoor Plant Delivered | Patch
Snake Plant
Index: 1
EPIPREMNUM GOLDEN POTHOS VASO 12CM – Peragashop
Pothos
Index: 2
If the user clicks on Snake Plant, your program would update:
state.selected_plant = 1
The number 1 represents the second plant in the list because Python starts counting at zero.
How is it used afterward?
Imagine the user wants to edit their Snake Plant.
Your program can use:
plant = state.plants[state.selected_plant]
This retrieves the plant at the selected index.
You can then modify its information, such as:
plant.water_frequency = 14
The same attribute can also be used when removing a plant:
state.plants.pop(state.selected_plant)
One important detail: You should only access the selected index after checking that it is valid. Also, after removing a plant, you should reset the selection or select another valid plant.
3. new_plant_name: str
Purpose: Temporarily stores the name that the user enters when adding a new plant.
This attribute is a little different from the other two because it stores information that the user types into an input box.
Imagine the user visits your Add Plant page.
Add a Plant
Plant Name
Enter plant name...
Sunlight Requirements
Enter sunlight hours...
Watering Frequency
Enter number of days...
When the user types "Monstera" into the Plant Name input, your State attribute would be updated:
state.new_plant_name = "Monstera"
The attribute temporarily holds the user's input until they click the Add button.
Your add_plant route can then use that information to create a new Plant dataclass.
For example:

```
new_plant = Plant(
    name=state.new_plant_name,
    # Other plant attributes...
)
```

After successfully adding the plant, you could clear the input:
state.new_plant_name = ""
This prevents the previous plant name from remaining in the form when the user wants to add another plant.
4. How all three attributes work together
Let's walk through a complete example.
User opens the website
current_page = "home"
selected_plant = -1
new_plant_name = ""
The user sees the home page. No plant has been selected.
User clicks Add a Plant
current_page = "add"
The website navigates to the Add Plant page.
User enters a plant name
new_plant_name = "Monstera"
The State stores the name entered into the input box.
User clicks Add
Your add_plant route creates a new Plant dataclass and adds it to state.plants.
new_plant_name = ""
current_page = "view_plants"
The user returns to their plant collection.
User selects the Monstera
selected_plant = 0
current_page = "plant_details"
The website displays the Monstera's information.
User clicks Edit Plant

current_page = "edit_plant"
The website opens the edit form for the selected Monstera.
5. My recommendation for your final State dataclass
Here is how I would organize your State attributes based on everything we've discussed:

```
@dataclass
class State:
    plants: list[Plant]
    current_page: str
    selected_plant: int
    new_plant_name: str
    search_query: str
```

I would initialize them with values similar to these:

```
State(
    plants=[],
    current_page="home",
    selected_plant=-1,
    new_plant_name="",
    search_query=""
)
```

Notice that I used -1 for selected_plant. This means no plant is currently selected.
One important thing to remember

Attribute	What it remembers	When it changes
current_page	Which page the user is viewing	When navigating
selected_plant	Which plant the user selected	When selecting or removing a plant
new_plant_name	The name being entered into the Add Plant form	When typing or submitting
plants	All the user's plant information	When adding, editing, or removing plants
search_query	What the user is searching for	When typing in the search bar

My biggest piece of advice: Don't think of State attributes as things that only belong to one page. They are information that your entire website can access. Different pages and route functions can read and modify the same State.
That is what allows your website to remember which plant someone selected, display its information, and then edit that same plant on another page!

Final Submission

When you are finished, download this file as a Word Document (docx) and upload the file to the appropriate assignment on Canvas.